

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph and Tree

Oleh:

Laily Azizah Islami

NIM. 2510817220020

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph and Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Laily Azizah Islami
NIM : 2510817220020

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	Error! Bookmark not defined.
DAFTAR ISI	ii
DAFTAR GAMBAR.....	iv
DAFTAR TABEL	v
SOAL 1: Konversi Adjacency List ke Adjacency Matrix.....	1
A. Source Code.....	3
B. Output Program	5
C. Pembahasan	5
SOAL 2: Konversi Adjacency Matrix ke Adjacency List.....	7
A. Source Code.....	9
B. Output Program	11
C. Pembahasan	11
SOAL 3: BFS: Jarak Terpendek Keluar Labirin	13
A. Source Code.....	15
B. Output Program	18
C. Pembahasan	18
SOAL 4: DFS: Menghitung Semua Jalur Keluar Labirin	20
A. Source Code.....	22
B. Output Program	24
C. Pembahasan	24
SOAL 5: Tree Traversal	26
A. Source Code.....	28
B. Output Program	31

C. Pembahasan	31
TAUTAN GIT	33

DAFTAR GAMBAR

Gambar 1. Screenshot Jawaban Soal 1	5
Gambar 2. Screenshot Jawaban Soal 2	11
Gambar 2. Screenshot Jawaban Soal 3	18
Gambar 4. Screenshot Jawaban Soal 4	24
Gambar 5. Screenshot Jawaban Soal 5	31

DAFTAR TABEL

Tabel 1. Source Code Soal 1	3
Tabel 2. Source Code Soal 2	9
Tabel 3. Source Code Soal 3	15
Tabel 4. Source Code Soal 4	22
Tabel 5. Source Code Soal 5	28

SOAL 1: Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U, V, W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U, V, W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

M

$U_1 V_1 W_1$

$U_1 V_2 W_2$

...

$U_M V_M W_M$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .

- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

$V_1 \ V_2 \ \dots \ V_N$

$V_1 \ a_{11} \ a_{12} \ \dots \ a_{1N}$

$V_2 \ a_{21} \ a_{22} \ \dots \ a_{2N}$

$\dots \ V_N \ a_{N1} \ a_{N2} \ \dots \ a_{NN}$

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N-1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki **self-loop**.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Penjelasan Contoh

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 D C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

Edge A B 4 menghasilkan nilai 4 pada baris A kolom B. Karena graph berarah, nilai pada baris B kolom A tetap 0 selama tidak ada edge dari B menuju A.

Edge D C 3 menghasilkan nilai 3 pada baris D kolom C. Vertex E tidak memiliki edge keluar, sehingga seluruh nilai pada baris E adalah 0.

A. Source Code

Tabel 1. Source Code Soal 1

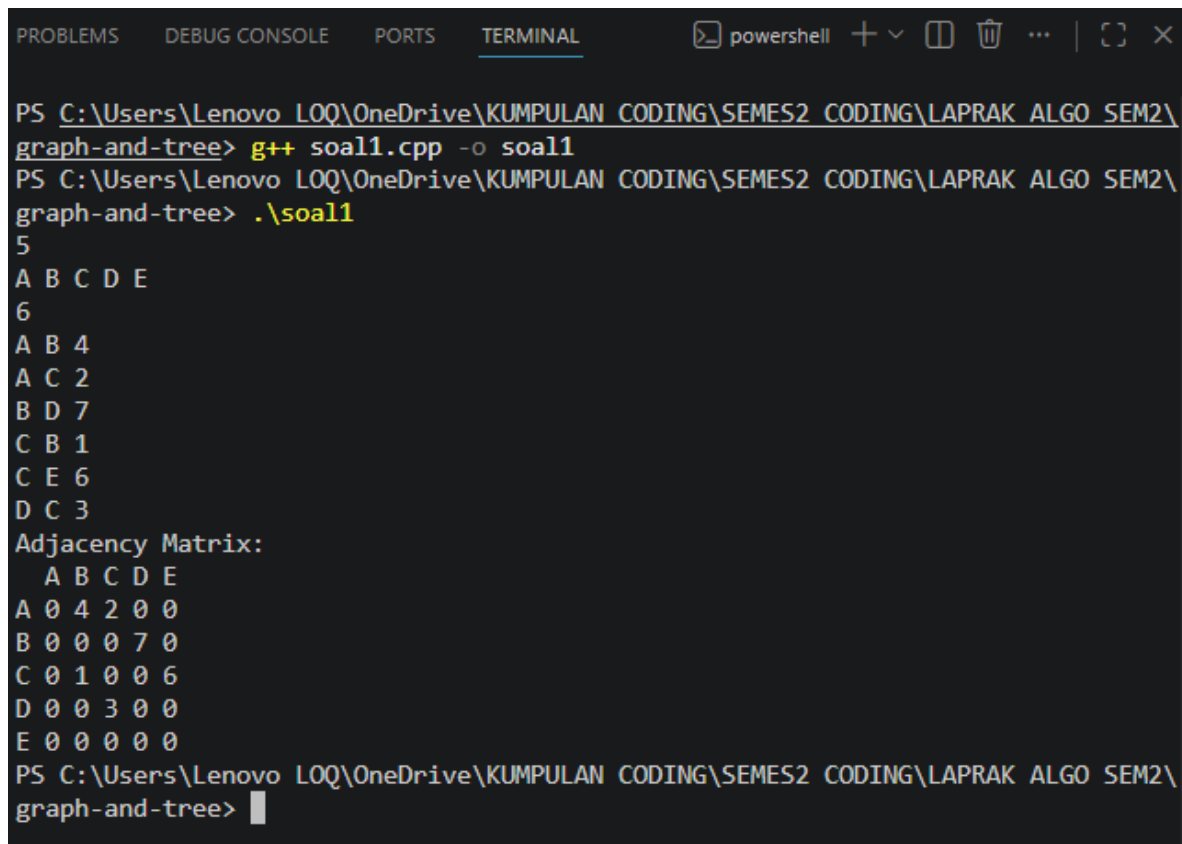
1	#include <iostream>
2	#include <vector>
3	#include <map>
4	
5	int main() {
6	int jumlahVertex, jumlahEdge;
7	std::cin >> jumlahVertex;
8	
9	std::vector<char> daftarVertex(jumlahVertex);
10	std::map<char, int> petaIndeks;
11	
12	for (int i = 0; i < jumlahVertex; ++i) {
13	std::cin >> daftarVertex[i];
14	petaIndeks[daftarVertex[i]] = i;
15	}
16	

```

17         std::vector<std::vector<int>>>
    matriksBobot(jumlahVertex,
    std::vector<int>(jumlahVertex, 0));
18     std::cin >> jumlahEdge;
19
20     for (int i = 0; i < jumlahEdge; ++i) {
21         char vertexU, vertexV;
22         int bobot;
23         std::cin >> vertexU >> vertexV >> bobot;
24         matriksBobot[petaIndeks[vertexU]][petaIndeks[ve
rtexV]] = bobot;
25     }
26
27     std::cout << "Adjacency Matrix:" << std::endl;
28     std::cout << " ";
29     for (int i = 0; i < jumlahVertex; ++i) {
30         std::cout << " " << daftarVertex[i];
31     }
32
33     std::cout << std::endl;
34     for (int i = 0; i < jumlahVertex; ++i) {
35         std::cout << daftarVertex[i];
36         for (int j = 0; j < jumlahVertex; ++j) {
37             std::cout << " " << matriksBobot[i][j];
38         }
39         std::cout << std::endl;
40     }
41
42     return 0;
43 }

```

B. Output Program



```
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK ALGO SEM2\graph-and-tree> g++ soal1.cpp -o soal1
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK ALGO SEM2\graph-and-tree> .\soal1
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
  A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK ALGO SEM2\graph-and-tree> █
```

Gambar 1. Screenshot Jawaban Soal 1

C. Pembahasan

1. Penjelasan Algoritma:

Program ini menggunakan algoritma representasi graf dengan menggunakan adjacency matrix. Adjacency matrix akan menyimpan graf dalam bentuk $n \times n$ dengan n yang merupakan jumlah vertex pada graf. Sedangkan nilai pada perpotongan baris dan kolom menunjukkan hubungan atau bobot edge yang menghubungkan kedua vertex.

Algoritma ini diawali dengan membaca jumlah vertex dan menyimpan setiap nama vertex pada sebuah array. Lalu program akan membuat pemetaan dengan menggunakan nama vertex dan indeks matrix dengan menggunakan struktur data map. Setelah seluruh vertex terbaca, program akan membentuk sebuah matrix yang diinisialisasi dengan nilai 0 (belum terdapat hubungan antar kedua vertex).

Kemudian, program akan membaca jumlah dari edge berserta hubungan antar kedua vertex. Sestiap kali membaca edge, program akan menentukan indeks kedua vertex melalui map, kemudian menyimpan bobot edge pada posisi matriks yang sesuai. Setelah proses algoritma selesai, program akan menampilkan isi matriks sebagai representasi dari graf.

2. Analisis:

Algoritma ini memiliki kompleksitas waktu dan ruang sebesar $O(V^2)$. Karena program akan mencetak seluruh elemen pada adjacency matriks yang berukuran $V \times V$. Meskipun terdapat proses pembacaan edge sebesar $O(E \log V)$, namun kompleksitas tersebut tidak mendominasi ketika vertex bertambah besar.

Keunggulan dari adjacency matriks ini adalah ia dapat mengakses dan mencari hubungan antara dua buah vertex secara cepat karena cukup melihat dari satu elemen matriks. Namun, algoritma ini kurang efisien untuk graf yang memiliki edge yang sedikit karena prosesnya tetap memerlukan ruang penyimpanan sebesar V^2 meskipun banyak elemen matriks yang bernilai 0.

Sehingga adjacency matriks cocok digunakan untuk graf yang memiliki jumlah hubungan vertex yang tergolong banyak dan membutuhkan akses data yang cepat terhadap keberadaan suatu edge.

SOAL 2: Konversi Adjacency Matrix ke Adjacency List

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang direpresentasikan dalam bentuk **adjacency matrix** berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi **adjacency list**. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

N V_1 V_2 ... V_N

A_{11} A_{12} ... A_{1N}

A_{21} A_{22} ... A_{2N}

...

A_{N1} A_{N2} ... A_{NN}

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

V1 -> (X1,w1), (X2,w2), ...

V2 -> (X1,w1), (X2,w2), ...

...

VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot

Contoh Kasus

Penjelasan Contoh

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B,4), (C,2) B -> (D,7) C -> (B,1), (E,6) D -> (C,3) E -> -

Pada baris vertex A, nilai positif muncul pada kolom B dan C. Karena itu, adjacency list untuk A adalah (B,4), (C,2).

Pada baris vertex E, semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah E -> -

A. Source Code

Tabel 2. Source Code Soal 2

```

1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      int jumlahVertex;
6      if (!(std::cin >> jumlahVertex)) return 0;
7
8      std::vector<char> daftarVertex(jumlahVertex);
9      for (int i = 0; i < jumlahVertex; i++) {
10         std::cin >> daftarVertex[i];
11     }
12
13         std::vector<std::vector<int>>
14     matriksBobot(jumlahVertex,
15         std::vector<int>(jumlahVertex));
16     for (int i = 0; i < jumlahVertex; i++) {
17         for (int j = 0; j < jumlahVertex; j++) {

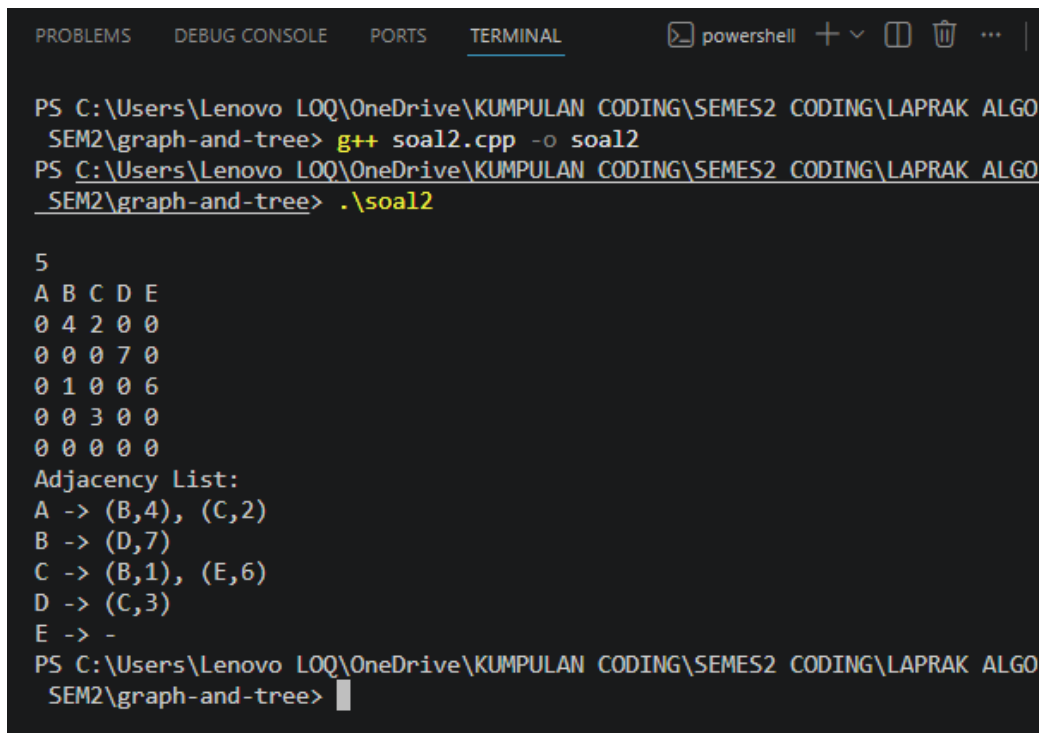
```

```

16         std::cin >> matriksBobot[i][j];
17     }
18 }
19
20     std::cout << "Adjacency List:" << std::endl;
21
22     for (int i = 0; i < jumlahVertex; i++) {
23         std::cout << daftarVertex[i] << " -> ";
24
25         bool terdapatEdge = false;
26
27         for (int j = 0; j < jumlahVertex; j++) {
28             if (matriksBobot[i][j] > 0) {
29                 if (terdapatEdge) std::cout << ", ";
30                 std::cout << "(" << daftarVertex[j] <<
",," << matriksBobot[i][j] << ")";
31                 terdapatEdge = true;
32             }
33         }
34
35         if (!terdapatEdge) {
36             std::cout << "-";
37         }
38
39         std::cout << std::endl;
40     }
41
42     return 0;
43 }

```


B. Output Program



```
PROBLEMS  DEBUG CONSOLE  PORTS  TERMINAL  powershell + - [ ] [ ] ... |
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK ALGO
SEM2\graph-and-tree> g++ soal2.cpp -o soal2
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK ALGO
SEM2\graph-and-tree> .\soal2

5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK ALGO
SEM2\graph-and-tree> 
```

Gambar 2. Screenshot Jawaban Soal 2

C. Pembahasan

1. Penjelasan Algoritma:

Program ini menggunakan algoritma yang mengkonversi adjacency matriks menjadi adjacency list untuk merepresentasikan sebuah graf berbobot. Adjacency list akan menyimpan daftar vertex yang terhubung dengan setiap vertex beserta bobot edge yang menghubungkannya.

Algoritma ini akan membaca jumlah dan nama vertex yang akan digunakan pada graf. Lalu, program akan membaca sebuah adjacency matrix yang berisi informasi mengenai hubungan antar vertex. Setiap elemen matriks akan menunjukkan bobot edge dari suatu vertex ke vertex lainnya. Jika nilai suatu matriks lebih besar dari 0, maka matriks tersebut akan dianggap memiliki hubungan antara kedua vertex tersebut.

Setelah seluruh data matriks terbaca, program akan melakukan penelusuran pada setiap baris matriks. Program akan memeriksa vertex pada seluruh kolom dan baris

yang terhubung. Jika ditemukan nilai yang lebih besar dari 0, maka vertex tujuan dan bobot edge akan ditambahkan ke dalam daftar adjacency list. Nilai 0 pada matriks akan menunjukkan bahwa tidak ada status hubungan antara kedua vertex tersebut. Sehingga program ini mengasumsikan bahwa graf yang digunakan merupakan graf berbobot dan terarah. Jika ditemukan nilai yang lebih besar dari 0 pada baris ke-I dan kolom ke-j maka program akan menambahkan vertex ke-j sebagai tetangga dari vertex ke-I berserta bobot edge yang sesuai. Sehingga, setiap hubungan yang ada pada adjacency matriks akan diubah menjadi adjacency list.

Pada akhir proses, program akan menampilkan hasil konversi dalam bentuk adjacency list, sehingga hubungan antar vertex dan terlihat seperti daftar yang lebih ringkas dibandingkan dengan representasi dari adjacency matriks. Sehingga algoritma ini dapat digunakan untuk menghasilkan adjacency list yang benar karena setiap elemen akan dicek satu per satu.

2. Analisis:

Kompleksitas waktu dan ruang pada algoritma ini adalah $O(V^2)$ dengan V yang menunjukkan total vertex pada graf. Kompleksitas ini didapatkan karena program harus membaca seluruh elemen adjacency matriks yang berukuran $V \times V$ serta melakukan penelusuran kembali terhadap seluruh elemen matriks saat membentuk adjacency list. Algoritma ini memiliki keunggulan yang mampu mengubah representasi adjacency matriks menjadi adjacency list, sehingga hubungan antar vertex ditampilkan dengan informasi yang lebih mudah dibaca.

SOAL 3: BFS: Jarak Terpendek Keluar Labirin

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma Breadth-First Search (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

R C G11 G12 ... G1C

G21 G22 ... G2C

...

GR1 GR2 ... GRC

SR SC

FR FC

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < R$
- $0 \leq FR < R$
- $0 \leq FC < R$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.

- Algoritma yang digunakan wajib BFS

Contoh Kasus

Input	Output
<pre> 8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9 </pre>	16

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (7,9). Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel (2,1).

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari (0,0) ke (7,9) adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Tabel 3. Source Code Soal 3

1	<code>#include <iostream></code>
2	<code>#include <vector></code>
3	<code>#include <queue></code>
4	
5	<code>struct point {</code>
6	<code> int baris, kolom, jarak;</code>
7	<code>};</code>
8	
9	<code>int main() {</code>

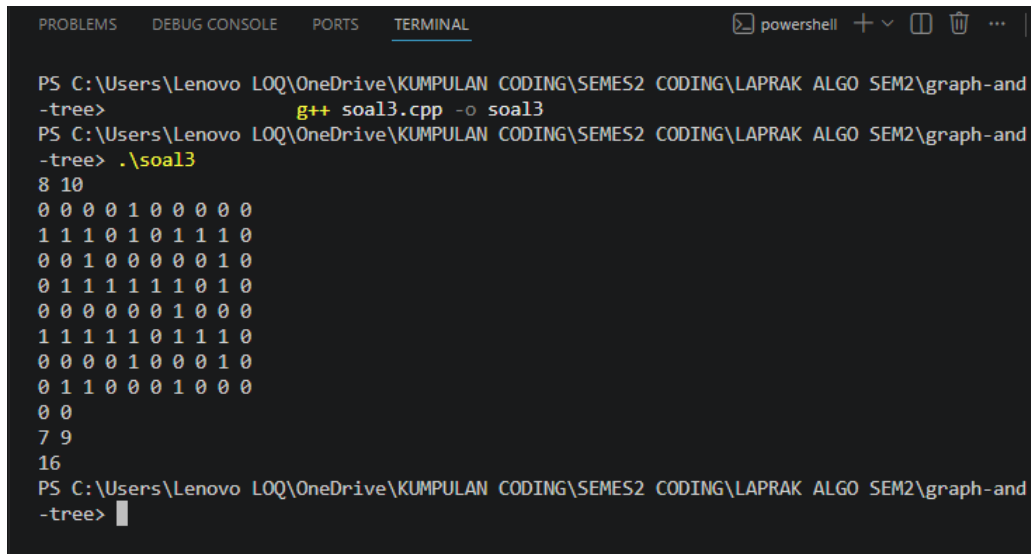
```

10     int jumlahBaris, jumlahKolom;
11     std::cin >> jumlahBaris >> jumlahKolom;
12
13     std::vector<std::vector<int>>> grid(jumlahBaris,
std::vector<int>(jumlahKolom));
14     for(int i = 0; i < jumlahBaris; ++i) {
15         for(int j = 0; j < jumlahKolom; ++j) {
16             std::cin >> grid[i][j];
17         }
18     }
19
20     int startBaris, startKolom, finishBaris, finishKolom;
21     std::cin >> startBaris >> startKolom >> finishBaris
>> finishKolom;
22
23     std::queue<point> antrian;
24     antrian.push({startBaris, startKolom, 0});
25     grid[startBaris][startKolom] = 1;
26
27     int arahBaris[] = {-1, 1, 0, 0};
28     int arahKolom[] = {0, 0, -1, 1};
29
30     bool ditemukan = false;
31     while(!antrian.empty()) {
32         point saatIni = antrian.front();
33         antrian.pop();
34
35         if(saatIni.baris == finishBaris && saatIni.kolom
== finishKolom) {
36             std::cout << saatIni.jarak;
37             ditemukan = true;

```

38	break;
39	}
40	
41	for(int i = 0; i < 4; ++i) {
42	int barisBaru = saatIni.baris + arahBaris[i];
43	int kolomBaru = saatIni.kolom + arahKolom[i];
44	
45	if(barisBaru >= 0 && barisBaru < jumlahBaris
	&& kolomBaru >= 0 && kolomBaru < jumlahKolom &&
	grid[barisBaru][kolomBaru] == 0) {
46	grid[barisBaru][kolomBaru] = 1;
47	antrian.push({barisBaru, kolomBaru,
	saatIni.jarak + 1});
48	}
49	}
50	}
51	
52	if(!ditemukan) {
53	std::cout << -1;
54	}
55	
56	return 0;
57	}

B. Output Program



```
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK ALGO SEM2\graph-and
-tree> g++ soal3.cpp -o soal3
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK ALGO SEM2\graph-and
-tree> .\soal3
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK ALGO SEM2\graph-and
-tree> |
```

Gambar 3. Screenshot Jawaban Soal 3

C. Pembahasan

1. Penjelasan Algoritma:

Program ini menggunakan algoritma Breadth First Search (BFS) yang akan bekerja dengan mencari jarak terpendek dari titik awal (start) hingga titik tujuan (finish) pada sebuah grid dengan dua dimensi. BFS merupakan algoritma yang melakukan penelusuran graf dengan mengunjungi seluruh simpul pada suatu tingkat sebelum berpindah ke tingkat selanjutnya. Karena setiap kali melakukan perpindahan memiliki bobot yang sama, algoritma ini dapat digunakan untuk menemukan jalur terpendek.

Cara kerja algoritma ini diawali dengan membaca ukuran grid dan isi dari setiap sel grid. Lalu untuk setiap angka 0 menunjukkan bahwa suatu sel dapat dilalui, sedangkan bila suatu sel menunjukkan nilai 1 berarti sel tersebut merupakan sebuah penghalang atau sel yang telah dikunjungi. Selanjutnya program akan membaca koordinat dari titik awal dan titik tujuan yang akan digunakan dalam proses pencarian jalur. Proses pencarian jalur dilakukan dengan menggunakan sistem antrian atau queue. Sehingga, titik awal yang dimasukkan akan diset dengan jarak awal 0, kemudian ditandai bahwa titik awal telah dikunjungi. Setiap kali melakukan iterasi,

program akan mengambil posisi di depan antrian dan memeriksa apakah posisi tersebut merupakan titik tujuan. Jika bukan maka program akan mengecek kemungkinan lainnya melalui arah pergerakan (atas, bawah, kiri, dan kanan).

Apabila posisi baru masih berada pada batas grid dan belum pernah dikunjungi, posisi tersebut akan ditandai telah dikunjungi dan dimasukkan pada antrian dengan jarak yang bertambah satu dari posisi sebelumnya. Proses ini terus dilakukan hingga titik tujuan ditemukan atau seluruh kemungkinan jalur telah diperiksa.

Program ini menggunakan ketentuan bahwa pergerakan hanya bisa dilakukan ke arah atas, bawah, kiri, dan kanan dengan biaya perpindahan yang sama pada setiap langkah. Oleh karena itu, BFS dapat digunakan untuk mencari jalur terpendek karena selalu memproses posisi yang terdekat. Jika titik tujuan ditemukan, jarak yang diperoleh merupakan jarak minimum. Sebaliknya, jika tujuan tidak dapat dicapai, program akan menghasilkan nilai -1.

2. Analisis:

Kompleksitas waktu dan ruang pada algoritma ini adalah $O(R \times C)$ dengan R menyatakan jumlah baris dan C menyatakan jumlah kolom pada grid. Hal ini dikarenakan setiap sel pada grid hanya diproses paling banyak satu kali selama pencarian berlangsung, serta program menyimpan data grid dan queue yang ukurannya dapat bertambah sebanding dengan jumlah sel pada grid.

SOAL 4: DFS: Menghitung Semua Jalur Keluar Labirin

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses backtracking agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

R C

G11 G12 ... G1C

G21 G22 ... G2C

...

GR1 GR2 ... GRC

SR SC

FR FC

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < R$
- $0 \leq FR < R$
- $0 \leq FC < R$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.

- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtrackin

Contoh Kasus

Input	Output
<pre> 5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5 </pre>	4

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (4,5). Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel (0,5).

Dengan DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

A. Source Code

Tabel 4. Source Code Soal 4

1	<code>#include <iostream></code>
2	
3	<code>using namespace std;</code>
4	
5	<code>int baris, kolom;</code>
6	<code>int grid[7][7];</code>
7	<code>bool cek[7][7];</code>
8	<code>int tBaris, tKolom;</code>

```

9   int jalur = 0;
10
11  int keBaris[] = {-1, 1, 0, 0};
12  int keKolom[] = {0, 0, -1, 1};
13
14  void cari(int b, int k) {
15      if (b == tBaris && k == tKolom) {
16          jalur++;
17          return;
18      }
19
20      cek[b][k] = true;
21
22      for (int i = 0; i < 4; i++) {
23          int bBaru = b + keBaris[i];
24          int kBaru = k + keKolom[i];
25
26          if (bBaru >= 0 && bBaru < baris &&
27              kBaru >= 0 && kBaru < kolom &&
28              grid[bBaru][kBaru] == 0 &&
29              !cek[bBaru][kBaru]) {
30              cari(bBaru, kBaru);
31          }
32      }
33
34      cek[b][k] = false;
35  }
36
37  int main() {
38      if (!(cin >> baris >> kolom)) return 0;
39

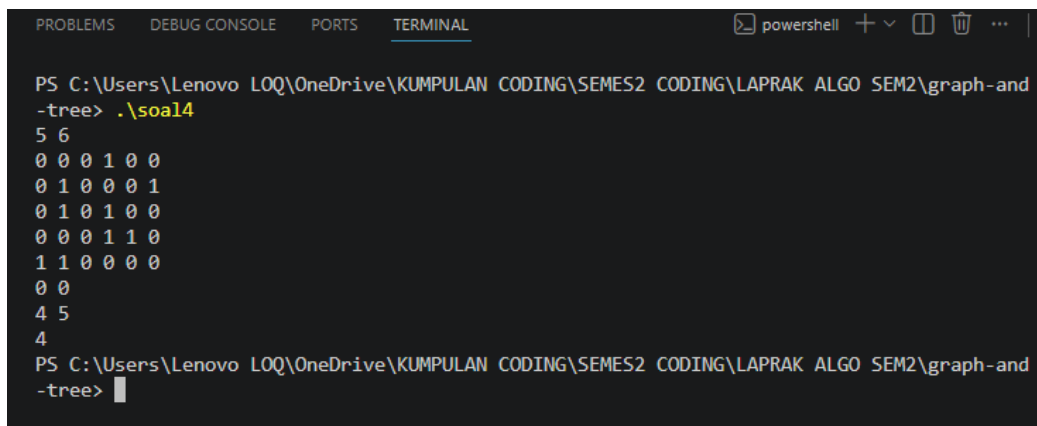
```

```

40     for (int i = 0; i < baris; i++) {
41         for (int j = 0; j < kolom; j++) {
42             cin >> grid[i][j];
43             cek[i][j] = false;
44         }
45     }
46
47     int bAwal, kAwal;
48     cin >> bAwal >> kAwal >> tBaris >> tKolom;
49
50     cari(bAwal, kAwal);
51
52     cout << jalur;
53
54     return 0;
55 }

```

B. Output Program



```

PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK ALGO SEM2\graph-and
-tree> .\soal4
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK ALGO SEM2\graph-and
-tree>

```

Gambar 4. Screenshot Jawaban Soal 4

C. Pembahasan

1. Penjelasan Algoritma:

Program ini menggunakan Depth First Search (DFS) dengan teknik backtracking untuk menghitung jumlah jalur yang dapat ditempuh dari titik awal menuju titik tujuan pada suatu grid. DFS bekerja dengan menelusuri satu jalur hingga jalur tersebut tidak dapat dilanjutkan lagi, kemudian kembali untuk mencoba jalur lainnya. Program akan membaca ukuran grid, isi grid, titik awal, dan titik akhir tujuan. Selanjutnya pada program ini terdapat fungsi `cari()` yang akan memeriksa arah pergerakan (atas, bawah, kiri, dan kanan). Jika posisi baru masih berada dalam batas grid, bukan penghalang, dan belum pernah dikunjungi, maka proses pencarian akan dilanjutkan secara rekursif. Ketika posisi tujuan telah ditemukan, variabel `jalur` akan ditambah satu. Setelah semua kemungkinan dari suatu posisi telah diperiksa, status kunjungan akan dikembalikan agar posisi tersebut dapat digunakan kembali untuk mengecek jalur yang berbeda. Sehingga, seluruh jalur yang valid dapat dihitung.

2. Analisis:

Kompleksitas waktu pada algoritma ini adalah $O(4^{(R \times C)})$ pada kondisi terburuk. Dengan R yang menyatakan jumlah baris, dan C yang menyatakan jumlah kolom pada grid. Hal ini dikarenakan setiap posisi, algoritma ini dapat mencoba seluruh kemungkinan arah pergerakan. Karena DFS akan berusaha untuk mengeksplor seluruh kemungkinan jalur yang dapat terbentuk dari titik awal hingga titik tujuan, jumlah proses yang dilakukan akan meningkat secara eksponensial bergantung pada ukuran grid.

Kompleksitas ruang pada algoritma ini adalah $O(R \times C)$. Hal ini disebabkan program menyimpan data grid dan array cek yang akan menandai sel yang telah dikunjungi. lalu proses rekursi pada DFS juga membutuhkan memori tambahan untuk menyimpan call stack. Pada kondisi terburuk call stack dapat mencapai sejumlah sel yang dilalui pada grid. oleh karena itu, kebutuhan ruang pada algoritma ini terus bertambah sebanding dengan ukuran grid yang digunakan.

SOAL 5: Tree Traversal

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah **Red Black Tree** secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTREE terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N

$A_1 A_2 \dots A_N$

Q

$T_1 T_2 \dots$

TQ

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTre.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar_angka

[Inorder] : daftar_angka

[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong,

cetak: Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$
- $-10000000000 \leq A_i \leq 10000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Penjelasan Contoh

Bilangan dimasukkan ke RBTre secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan.

Hasil INORDER pada RBTree selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTree yang dibentuk sudah konsisten dengan aturan RBTree.

A. Source Code

Tabel 5. Source Code Soal 5

1	<code>#include <iostream></code>
2	<code>#include <string></code>
3	<code>#include "RedBlackTree.h"</code>
4	
5	<code>using namespace std;</code>
6	
7	<code>void preOrder(const RedBlackTree::Node* simpul, const</code>
	<code>RedBlackTree::Node* nil) {</code>
8	<code> if (simpul == nil) return;</code>
9	
10	<code> cout << simpul->key << " ";</code>
11	<code> preOrder(simpul->left, nil);</code>
12	<code> preOrder(simpul->right, nil);</code>
13	<code>}</code>
14	
15	<code>void inOrder(const RedBlackTree::Node* simpul, const</code>
	<code>RedBlackTree::Node* nil) {</code>
16	<code> if (simpul == nil) return;</code>
17	
18	<code> inOrder(simpul->left, nil);</code>
19	<code> cout << simpul->key << " ";</code>
20	<code> inOrder(simpul->right, nil);</code>
21	<code>}</code>
22	

```

23 void postOrder(const RedBlackTree::Node* simpul, const
    RedBlackTree::Node* nil) {
24     if (simpul == nil) return;
25
26     postOrder(simpul->left, nil);
27     postOrder(simpul->right, nil);
28     cout << simpul->key << " ";
29 }
30
31 int main() {
32     int banyakData;
33     cin >> banyakData;
34
35     RedBlackTree tree;
36
37     for (int i = 0; i < banyakData; i++) {
38         int angka;
39         cin >> angka;
40
41         if (!tree.contains(angka)) {
42             tree.insert(angka);
43         }
44     }
45
46     if (tree.empty()) {
47         cout << "Tree kosong. Tidak ada yang bisa
ditampilkan.";
48         return 0;
49     }
50
51     int banyakQuery;

```

```

52     cin >> banyakQuery;
53
54     string daftarQuery[20];
55
56     for (int i = 0; i < banyakQuery; i++) {
57         cin >> daftarQuery[i];
58     }
59
60     for (int i = 0; i < banyakQuery; i++) {
61         if (daftarQuery[i] == "PREORDER") {
62             cout << "[Preorder]: ";
63             preOrder(tree.root(), tree.nil());
64             cout << endl;
65         }
66         else if (daftarQuery[i] == "INORDER") {
67             cout << "[Inorder]: ";
68             inOrder(tree.root(), tree.nil());
69             cout << endl;
70         }
71         else if (daftarQuery[i] == "POSTORDER") {
72             cout << "[Postorder]: ";
73             postOrder(tree.root(), tree.nil());
74             cout << endl;
75         }
76         else if (daftarQuery[i] == "ALL") {
77             cout << "[Preorder]: ";
78             preOrder(tree.root(), tree.nil());
79             cout << endl;
80
81             cout << "[Inorder]: ";
82             inOrder(tree.root(), tree.nil());

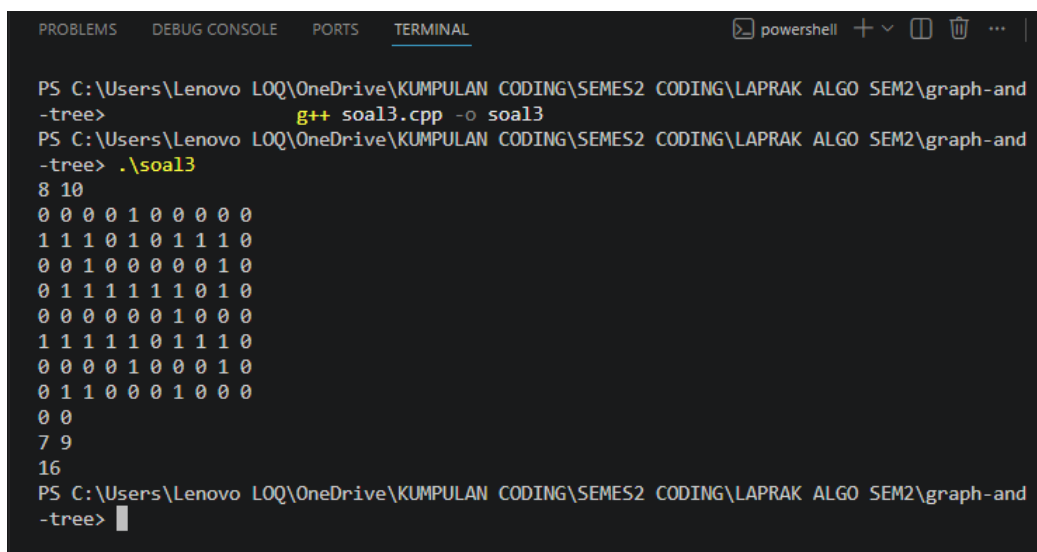
```

```

83         cout << endl;
84
85         cout << "[Postorder]: ";
86         postOrder(tree.root(), tree.nil());
87         cout << endl;
88     }
89 }
90
91 return 0;
92 }

```

B. Output Program



```

PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK ALGO SEM2\graph-and
-tree> g++ soal3.cpp -o soal3
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK ALGO SEM2\graph-and
-tree> .\soal3
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
PS C:\Users\Lenovo LOQ\OneDrive\KUMPULAN CODING\SEMES2 CODING\LAPRAK ALGO SEM2\graph-and
-tree>

```

Gambar 5. Screenshot Jawaban Soal 5

C. Pembahasan

1. Penjelasan Algoritma:

Program ini menggunakan struktur data Red-Black-Tree merupakan algoritma yang memiliki mekanisme penyeimbangan otomatis dengan menjaga proses pencarian dan penyisipan data dengan efisien dibanding metode BST biasa.

Sebelum membaca data, program akan memeriksa terlebih dahulu apakah data yang dimasukkan sudah menggunakan fungsi `contains()`. Jika belum, data akan dimasukkan terlebih dahulu dengan menggunakan fungsi `insert()`. Jika semua data telah tersimpan, program akan membaca perintah yang menentukan jenis traversal yang akan ditampilkan (Preorder, inorder, atau postorder).

2. Analisis:

Kompleksitas waktu pada saat program melakukan pencarian dan penyisipan data adalah $O(\log n)$, jika terdapat data sebanyak n data yang dimasukkan, maka proses pembentukan pohon akan menggunakan kompleksitas waktu $O(n \log n)$ karena setiap data harus melewati proses pencarian dan penyisipan.

Pada proses traversal setiap simpul akan dikunjungi tepat satu kali, sehingga kompleksitas waktu yang diperlukan adalah $O(n)$. Lalu kompleksitas ruang algoritma pada Red-Black-Tree yang diperlukan sebesar $O(n)$ karena semakin banyak data yang dimasukkan, semakin besar pula memori yang dibutuhkan untuk menyimpan data pohon.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-laily-gie.git>